Stundendokumentation Mittwoch, 04.11.2026: `Vertex` lernt rechnen und vergleichen

Informatik LK, 1./2. Stunde | Einheit 2 „Objektorientierung und Spielprojekt“ | Buch „Java will nur spielen“, Abschnitt 3.1.4 und 3.1.5

Was wir gemacht haben

1. **Ankommen:** Fragen zur Klausur geklärt, Plan der Stunde.
2. **Aufgabe 1:** Methode `length()` für den Abstand vom Ursprung. Besprochen: Warum gibt es kein Feld `laenge`?
3. **Aufgabe 2 zu zweit in BlueJ:** vier Methoden, je ein Paar aus „neuer Punkt“ und „veränderter Punkt“ (`skalarMult`, `skalarMultMod`, `add`, `addMod`). Die Tabelle in `TestVertex` wurde **vor** dem Start ausgefüllt. Besprochen: Warum ist die verändernde Variante riskanter? Beispiel: Zwei Spielfiguren teilen sich einen Punkt, eine bewegt sich.
4. **Aufgabe 3:** `x = x;` im Setter, zuerst falsch, dann mit `this`. Dazu Getter und Setter.
5. **Aufgabe 4:** `==` und `equals`. Das Beispiel mit den Strings, `equals` für `Vertex` mit `instanceof` und Cast, die Tabelle mit den Vergleichen und die bewusst provozierte `ClassCastException`.
6. **Aufgabe 5:** `Vertex` als UML-Klassendiagramm auf Papier. Zum Vergleich: BlueJ zeigt nur die Klasse, nicht ihre Felder und Methoden.
7. **Schmiede:** Unit 60 „E2 Das Spiel“, L2 „Vertex: Methoden und equals“. Wer L1 noch nicht hatte, begann dort.
8. **Sicherung:** Merke-Tabelle vom Blatt. Ausblick: Donnerstag geht es um geometrische Figuren (Buch 3.2).

Wenn du gefehlt hast

1. Hol dir das Blatt **„Vertex lernt rechnen und vergleichen“** aus dem Gruppenordner KW45 und öffne dein Projekt `spiel` mit der Klasse `Vertex` vom 03.11.
2. Bearbeite **Aufgabe 1** und **Aufgabe 2**. Füll die Tabelle **vor** dem Start von `TestVertex` aus.
3. Bearbeite **Aufgabe 3** und **Aufgabe 4**. Teste in 3a wirklich die falsche Fassung `x = x;`.
4. Zeichne in **Aufgabe 5** das Klassendiagramm auf Papier.
5. Arbeite danach die Schmiede-Lektion U60 L2 durch und lies erst dann den Erwartungshorizont. Übertrage den Merke-Kasten ins Heft.

Erwartungshorizont

Alle Ausgaben stammen aus Läufen mit `javac` und `java` 25.

Aufgabe 1: Wie weit ist es zum Ursprung?

```
public double length() {
    return Math.sqrt(x * x + y * y);
}
```

`new Vertex(3, 4).length()` ergibt `5.0`. In Java ist `x^2` kein Quadrat, sondern das bitweise Exklusiv-Oder. Nimm `x * x` oder `Math.pow(x, 2)`.

Für den Abstand gibt es kein eigenes Feld, weil er von `x` und `y` abhängt. Ein Feld `laenge` müsste nach jeder Änderung von `x` oder `y` neu gesetzt werden, sonst wäre es veraltet. Was sich berechnen lässt, bekommt eine Methode.

Aufgabe 2: Neuer Punkt oder veränderter Punkt?

```java
public Vertex skalarMult(double s) {
    return new Vertex(x * s, y * s);
}

public void skalarMultMod(double s) {
    x = s * x;
    y = s * y;
}

public Vertex add(Vertex v2) {
    return new Vertex(x + v2.x, y + v2.y);
}

public void addMod(Vertex v2) {
    x = x + v2.x;
    y = y + v2.y;
}
```

Mit `Vertex v1 = new Vertex(3, 4);` und `Vertex v2 = new Vertex(1, 2);`:

Zeile	Ausgabe
`System.out.println(v1.skalarMult(2));`	`(6.0,8.0)`
`System.out.println(v1);`	`(3.0,4.0)`
`v1.skalarMultMod(2);`	keine Ausgabe
`System.out.println(v1);`	`(6.0,8.0)`
`System.out.println(v1.add(v2));`	`(7.0,10.0)`
`System.out.println(v1);`	`(6.0,8.0)`
`v1.addMod(v2);`	keine Ausgabe
`System.out.println(v1);`	`(7.0,10.0)`
`System.out.println(v2);`	`(1.0,2.0)`

Die Pointe steht in Zeile 2 und 6: `skalarMult` und `add` lassen `v1` unverändert. Häufige Fehler: `return` in einer `void`-Methode und ein Rückgabewert, der nicht ausgegeben wird. Eine Methode `skalarMult` mit `void` kann nichts liefern, das `println` ausgeben könnte.

Warum ist die verändernde Variante gefährlicher? Alle anderen Stellen, die dasselbe Objekt kennen, bekommen die Änderung mit, ohne dass sie dort im Code steht. In tausend Zeilen findet man kaum, wer den Punkt verschoben hat.

Aufgabe 3: `this`, Getter und Setter

3a) `x = x;` weist dem Parameter `x` seinen eigenen Wert zu. Das Feld bleibt unverändert. Nach `setX(5)` zeigt der Punkt weiter den alten x-Wert. Der Compiler meldet nichts, der Fehler fällt erst beim Testen auf.

3b)

```
public void setX(double x) { this.x = x; }
public void setY(double y) { this.y = y; }
public double getX() { return x; }
public double getY() { return y; }
```

In den Gettern ist `this.` nicht nötig, weil kein Parameter das Feld verdeckt. Mit `this.x` ist es ebenfalls richtig. Ändert sich nach der Korrektur nichts, hilft es, in BlueJ neu zu übersetzen und alte Objekte von der Objektleiste zu entfernen.

Aufgabe 4: Das Gleiche oder dasselbe?

4a) `false` und `true`. `toLowerCase()` erzeugt zur Laufzeit ein neues String-Objekt: gleicher Inhalt, anderes Objekt. `==` fragt „dasselbe Objekt?“, `equals` fragt „gleich?“.

4b)

```
public boolean equals(Object thatObject) {
    if (thatObject instanceof Vertex) {
        Vertex that = (Vertex) thatObject;
        return this.x == that.x && this.y == that.y;
    }
    return false;
}
```

Der Parameter muss den Typ `Object` haben. Mit `equals(Vertex that)` übersetzt die Klasse zwar, aber sie überschreibt das `equals` von `Object` nicht: Sobald ein Punkt als `Object` gesehen wird, etwa bei `contains` in einer Liste, wird wieder nur auf dasselbe Objekt geprüft.

4c) Mit `v2 = new Vertex(1, 2)` und `v3 = new Vertex(1, 2)`:

Ausdruck	Ergebnis
`v2 == v3`	`false`
`v2.equals(v3)`	`true`
`v2.equals("hallo")`	`false`

4d) Ein **Laufzeitfehler**, eine `ClassCastException`: Ein `String` lässt sich nicht in `Vertex` umwandeln. Der Compiler kann das nicht wissen, weil der Parameter den Typ `Object` hat und jeder Aufruf erlaubt ist. Der Cast verspricht ihm einen Typ, den erst die Laufzeit prüft.

Aufgabe 5: `Vertex` als UML-Klassendiagramm

```
+-----------------------------------------+
|                 Vertex                  |
+-----------------------------------------+
| + x : double                            |
| + y : double                            |
+-----------------------------------------+
| + Vertex(xx : double, yy : double)      |
| + length() : double                     |
| + skalarMult(s : double) : Vertex       |
| + add(v2 : Vertex) : Vertex             |
| + setX(x : double) : void               |
| + getX() : double                       |
| + equals(thatObject : Object) : boolean |
| + toString() : String                   |
+-----------------------------------------+
```

Erwartet sind drei Fächer (Name, Felder, Methoden), `+` für `public`, der Typ hinter dem Doppelpunkt und der Konstruktor ohne Rückgabetyp. Pflicht sind Felder, Konstruktor, `length`, `add` und `equals`. Weitere Methoden dürfen dazu.

Zusatz

```
public Vertex sub(Vertex that) {
    return new Vertex(x - that.x, y - that.y);
}

public double distance(Vertex that) {
    return sub(that).length();
}

public Vertex normalized() {
    return skalarMult(1 / length());
}
```

`new Vertex(0, 0).distance(new Vertex(6, 8))` ergibt `10.0`. `new Vertex(6, 8).normalized()` liefert `(0.6000000000000001,0.8)`. Das ist Rundung bei `double`, der Code stimmt.

Merke

Begriff	Bedeutung
Methode	Arbeitet mit den Feldern des Objekts, auf dem sie aufgerufen wird
`this`	Das Objekt, für das die Methode gerade läuft. `this.x` ist das Feld, `x` allein ist hier der Parameter
`==`	Prüft bei Objekten, ob es **dasselbe** Objekt ist
`equals`	Prüft, ob zwei Objekte **gleich** sind. Was das heißt, legst du selbst fest
`instanceof`	Fragt, ob ein Objekt zu einer Klasse gehört
Cast	`(Vertex) o` sichert dem Compiler einen Typ zu. Stimmt er nicht, stürzt das Programm zur Laufzeit ab